

Automates et Grammaires

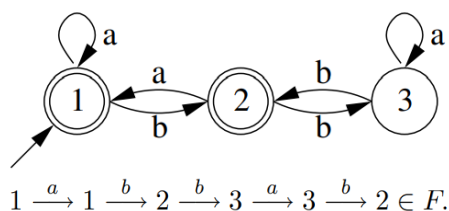
David Darras, 19 décembre 2023

1 Automates, Langages

1.1 Définitions

- Un **alphabet** Σ est un ensemble fini de symboles/lettres.
Ex: $\Sigma_1 = \{a\}$, $\Sigma_2 = \{\heartsuit, \diamondsuit, \clubsuit, \spadesuit\}$, $\Sigma_3 = \{0, 1\}$
- Un **mot** m est une séquence (= suite finie et ordonnée) de symboles. On note le **mot vide** ε et vérifie : $m \cdot \varepsilon = \varepsilon \cdot m = m$ et $|\varepsilon| = 0$.
Ex: $m_1 = \text{dabac}$, $m_2 = 000101$, $m_3 = (()())$
- Un **automate fini (AEF)** est défini par un quintuplet $(Q, \Sigma, \delta, q_0, F)$, où :
 - Q est un ensemble d'états.
 - Σ est l'alphabet.
 - δ est la fonction de transition qui spécifie comment l'automate passe d'un état à un autre.
 - q_0 est l'**état initial** à partir duquel l'automate démarre, noté X_i .
 - F est l'ensemble d'états **accepteurs/finaux**, notés X_a .

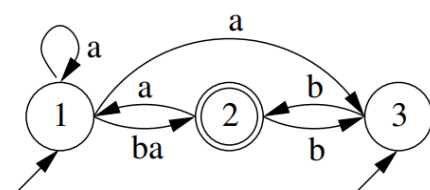
En bref, un automate désigne une machine qui reconnaît des mots. Il peut être représenté à l'aide d'un graphe et décrit par une **table de transitions**.



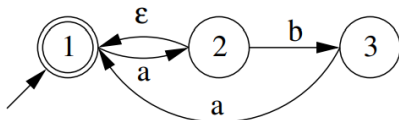
$$1 \xrightarrow{b} 2 \xrightarrow{b} 3 \xrightarrow{a} 3 \notin F.$$

	$1_{i,a}$	2_a	3
a	1	1	3
b	2	3	2

- Un **langage** L sur Σ est un ensemble de mots sur Σ . Lorsque celui-ci est reconnu par un automate sur Σ , on le note $\mathcal{L}(A)$.
Ex: $L_1 = \{\varepsilon, a, bb, abc, ef\}$, $L_2 = \{a^n \mid n \in \mathbb{N}\}$
- On dit que deux automates sont **équivalents** lorsqu'ils acceptent le même langage.
- Une **expression régulière** est une représentation textuelle d'un automate.
Ex: $\mathcal{L}(a^*bc)$, $\mathcal{L}((a|b)c^*)$
- Un automate fini est dit **déterministe** s'il possède un **seul état initial** et que, d'un même état, ne peuvent jamais être issues deux transitions différentes étiquetées par la même lettre.
- On reconnaît un **automate fini non déterministe (AFND)** (ou asynchrone) lorsqu'on a plusieurs états initiaux ou plusieurs choix pour une même transition.
Ex: En partant de 1 et en lisant un a, on peut soit aller dans 1, soit dans 3.



- De plus, on peut y ajouter des transitions "vides", notées ε -**transitions**, qui permettent de sauter d'un état à un autre sans avoir rien lu.

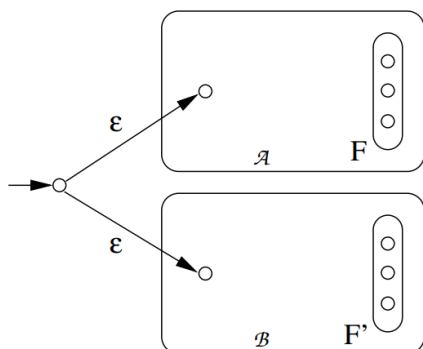


1.2 Opérateurs sur les mots

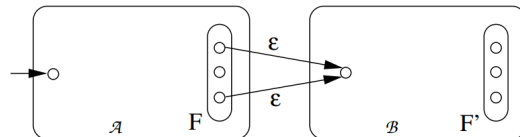
- **Concaténation:** $ab \cdot cd = abcd = \varepsilon \cdot a \cdot \varepsilon \cdot bcd$.
- **Projection:** $(abcba)_{/a} = aa$
- **Longueur:** $|abcd| = 4$ et $|\varepsilon| = 0$
- **Itération:** $(abc)^3 = abc \cdot abc \cdot abc$

1.3 Opérateurs sur les langages

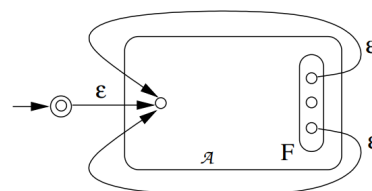
- **Union:** $L_1 \cup L_2 = \{x \mid x \in L_1 \vee x \in L_2\}$
NB: L'union correspond à la somme de deux automates.



- **Intersection:** $L_1 \cap L_2 = \{x \mid x \in L_1 \wedge x \in L_2\}$
NB: L'intersection correspond au produit de deux automates.
- **Différence:** $L_1 \setminus L_2 = \{x \mid x \in L_1 \wedge x \notin L_2\}$
- **Complémentaire:** $\mathcal{L}(A^c) = \overline{\mathcal{L}(A)} = \{x \mid x \notin L\} = \Sigma^* - \mathcal{L}(A)$.
- **Concaténation:** $L_1 \cdot L_2 = \{x \cdot y \mid x \in L_1, y \in L_2\}$
Attention: $\{\} \cdot L = \{\}$, $\{\varepsilon, a\} \cdot \{b, \varepsilon\} = \{b, \varepsilon, ab, a\}$. On appelle $\{\}$ l'**élément absorbant** et $\{\varepsilon\}$ l'**élément neutre** pour la concaténation.
Remarque: La concaténation est distributive pour l'union et l'intersection.



- **Itération:** $L^n = \{x_1 \cdot \dots \cdot x_n \mid (x_1, \dots, x_n) \in L \times \dots \times L\}$
Exemple: $\{a, b\}^2 = \{aa, ab, ba, bb\}$, $L^0 = \{\varepsilon\}$, $\forall n \geq 1, \{\}^n = \{\}$
NB: Pour construire l'automate résultant, il suffit de chaîner les automates avec ε -transitions.
- **Étoile de Kleene:** (=fermeture itérative) $L^* = \bigcup_{i \in \mathbb{N}} L^i = \{x_1 \cdot x_2 \cdot \dots \cdot x_k \mid k \geq 0, (x_1, \dots, x_k) \in L \times \dots \times L\}$
Remarques: $\varepsilon \in L$ et $L \subseteq \Sigma^*$
NB: Pour construire l'automate résultant, il suffit de boucler avec une ε -transition sur un état pré-initial lui-même lié avec une ε -transition à l'état initial.



1.4 Opérateurs sur les expressions régulières

- $\mathcal{L}(0) = \{\}$ → (start state, no transitions)
- $\mathcal{L}(.) = \Sigma$ → (start state, self-loop on all symbols)
- $\mathcal{L}(\sigma) = \{\sigma\}$ → (start state, transition on symbol σ to final state)
- $\mathcal{L}(\$) = \{\varepsilon\}$ → (start state, self-loop on ε to final state)
- $\mathcal{L}(e_1 \cdot e_2) = \mathcal{L}(e_1) \cdot \mathcal{L}(e_2)$
- $\mathcal{L}(e_1 \mid e_2) = \mathcal{L}(e_1) \cup \mathcal{L}(e_2)$
- $\mathcal{L}(e_1 \& e_2) = \mathcal{L}(e_1) \cap \mathcal{L}(e_2)$
- $\mathcal{L}(e^*) = \mathcal{L}(e)^*$

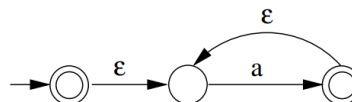


FIGURE III.5. AFND acceptant $\{a\}^*$.

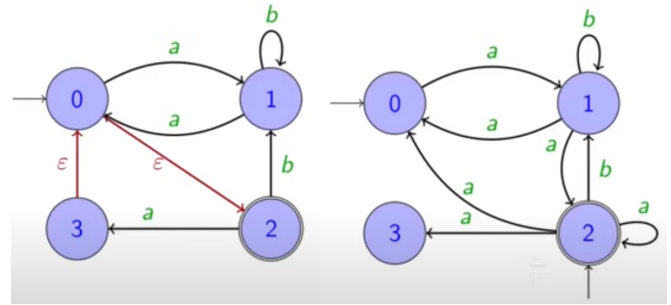
- $\mathcal{L}(e+) = \mathcal{L}(e \cdot e^*)$

- $\mathcal{L}(\tilde{e}) = \overline{\mathcal{L}(e)}$

- Exemples

- $\mathcal{L}(a^*b^*) = \mathcal{L}(a^*) \cdot \mathcal{L}(b^*) = \mathcal{L}(a)^* \cdot \mathcal{L}(b)^* = \{a\}^* \cdot \{b\}^* = \{a^i \mid i \in \mathbb{N}\} \cdot \{b^j \mid j \in \mathbb{N}\} = \{a^i \cdot b^j \mid (i, j) \in \mathbb{N}^2\}$
- $\mathcal{L}((a \mid b)^*) = \mathcal{L}((a^* \cdot b^*)^*)$

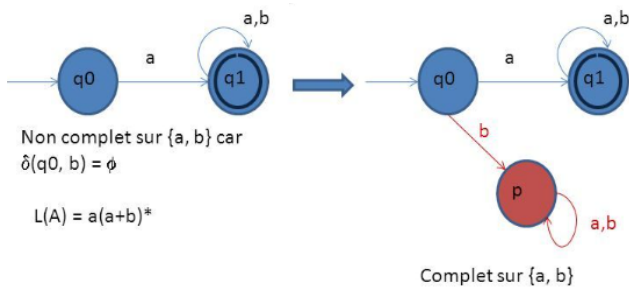
- $(*, +)$ sont prioritaires sur $(\cdot)$ qui est lui-même prioritaire sur $(\mid, \&)$.



1.5 Algorithmes à retenir

1.5.1 Compléter un automate

1. Créer un état puits.
2. Générer toutes les transitions possibles et manquantes vers cet état.
3. Remarque: Un automate est complet ssi $\text{card}(\delta) = \text{card}(A) \times \text{card}(Q)$



1.5.2 Éliminer les ε -transitions

NB: Cette méthode rend l'automate fortement non déterministe et ne génère aucun nouvel état.

1. Réaliser la clôture transitive des ε -transitions.
(1) – ε – > (2) – ε – > (3) donne (1) – ε – > (3)
2. Tracer les raccourcis : (1) – ε – > (2) – a – > (3) donne (1) – a – > (3).
3. Propager les états accepteurs en arrière : (1) – b – > (2) – ε – > ((3)) donne (1) – b – > ((2))
4. Propager les états initiaux en avant : – > (1) – ε – > (2) – a – > (3) donne – > (1) – a – > (3)
5. Supprimer les ε -transitions
6. Vidéo explicative

1.5.3 Déterminiser un automate - Construction par sous-ensembles

- Rappel du lemme de Rabin et Scott : Tout langage accepté par un AFND est accepté par un AFD.
- Soit $A = (\Sigma, Q, D, F, \delta)$ non déterministe. Alors son automate déterministe équivalent $A_d = (\Sigma, Q_d, D_d, F_d, \delta_d)$ vérifie:

- $Q_d \subseteq 2^Q$

Ex: Pour $Q = \{0, 1\}$, on aura les états suivants: $\emptyset, \{0\}, \{1\}, \{0, 1\}$ pour A_d

Remarque: on peut supprimer les états qui ne sont pas accessibles.

- $D_d = \{D\}$

Ex: Si 3, 5 et 6 sont les états initiaux de A alors $\{3, 5, 6\}$ sera l'unique état initial.

- $F_d = \{P \in Q_d, P \cap F \neq \emptyset\}$ = Ensemble des états contenant au moins un état final
Ex: Pour $Q = \{0, 1\}$. Si 0 est un état final alors $\{0\}$ et $\{0, 1\}$ seront les deux états finaux de A_d

- Pour dessiner les transitions, il suffit de faire tourner simultanément plusieurs machines et de mettre les états dans lesquels on tombe sous forme d'ensemble.

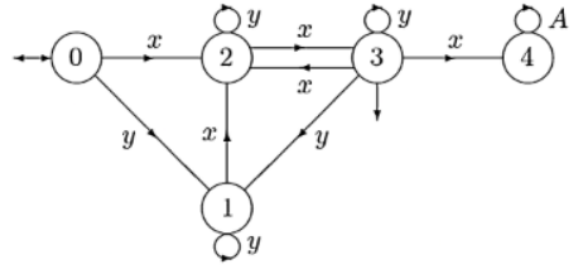
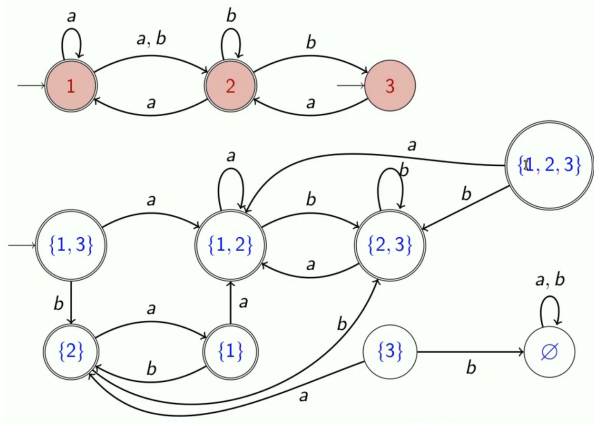
Ex: (cf. figure ci-dessous) :

On part de 1. Si on lit un a , on peut soit aller dans 1, soit dans 2. On va donc partir de $\{1\}$ dans A_d et aller dans $\{1, 2\}$. Et on fait pareil pour toutes les autres lettres. Si on veut faire les transitions de $\{1, 2\}$, on regarde dans quel état on tombe si on lit un a à partir de 1, puis 2 et on met tous les états sous forme d'ensemble. Ici $(1) - a - > (1, 2)$ et $(2) - a - > (1)$ donc $(1, 2) - a - > (1, 2)$

- Vidéo explicative n°1

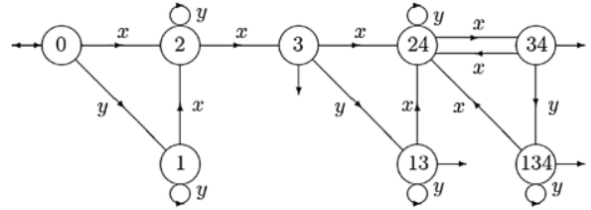
- Vidéo explicative n°2

• Exemple 1

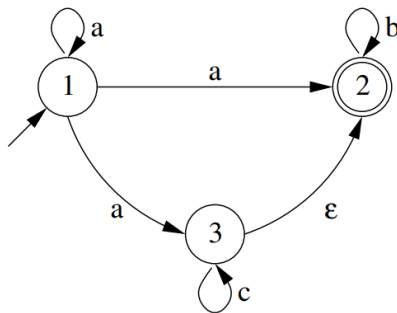


δ	0	1	2	3	4
x	2	2	3	{2,4}	4
y	1	1	2	{1,3}	4

δ_d	{0}	{2}	{1}	{3}	{2,4}	{1,3}	{3,4}	{1,3,4}
x	{2}	{3}	{2}	{2,4}	{3,4}	{2,4}	{2,4}	{2,4}
y	{1}	{2}	{1}	{1,3}	{2,4}	{1,3}	{1,3,4}	{1,3,4}

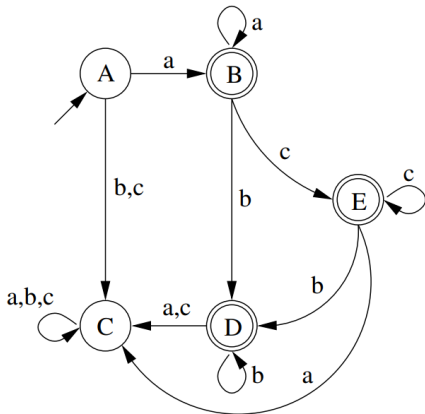


• Exemple 2



q	q.a	q.b	q.c
1	{1,2,3}		
2		2	
3		2	{2,3}

X	X.a	X.b	X.c
{1}	{1,2,3}	$\emptyset$	$\emptyset$
{1,2,3}	{1,2,3}	{2}	{2,3}
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
{2}	$\emptyset$	{2}	$\emptyset$
{2,3}	$\emptyset$	{2}	{2,3}



On pose $A = \{1\}$, $B = \{1, 2, 3\}$, $C = \emptyset$, $D = \{2\}$ et $E = \{2, 3\}$.

• Exemple 3

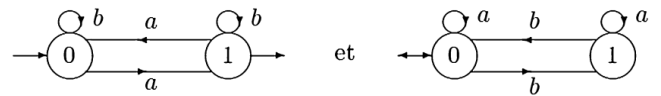
1.5.4 Somme de deux automates $A + B$

- Soient $A = (\Sigma, Q_A, D_A, F_A, \delta_A)$ et $B = (\Sigma, Q_B, D_B, F_B, \delta_B)$. Alors $A + B = (\Sigma, Q_A \cup Q_B, D_A \cup D_B, F_A \cup F_B, \delta_A \cup \delta_B)$

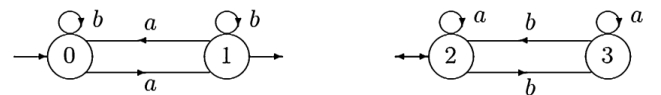
- Il suffit donc d'accoler les deux tables de transitions. On aura alors un automate avec deux parties connexes (i.e., deux parties qui ne sont liées par aucune transition).

Remarque: On peut faire le choix de créer un unique état initial et d'utiliser deux ϵ -transitions.

Pour ces deux automates:



On obtient:

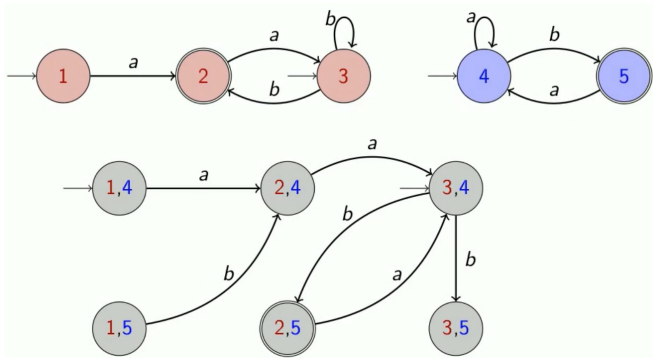


1.5.5 Produit de deux automates $A \times B$

- Soient $A = (\Sigma, Q_A, \{q_{0A}\}, F_A, \delta_A)$ et $B = (\Sigma, Q_B, \{q_{0B}\}, F_B, \delta_B)$. Alors $A \times B = (\Sigma, Q_A \times Q_B, \{(q_{0A}, q_{0B})\}, F_A \times F_B, \delta)$ où $\delta = \{((p_A, p_B), \sigma, (q_A, q_B)) \mid (p_A, \sigma, q_A) \in \delta_A, (p_B, \sigma, q_B) \in \delta_B\}$

- Remarque 1: On a au plus $\text{Card}(A) \times \text{Card}(B)$ couples d'états pour l'automate produit.

- Remarque 2: $f = (f_A, f_B)$ est un état acceptant ssi f_1 et f_2 sont des états acceptants.
- L'idée est de parcourir simultanément les deux automates et de compléter les transitions au fur et à mesure. Pour ce faire, utilise la méthode des jetons : <https://www.youtube.com/watch?v=buovniusmw>



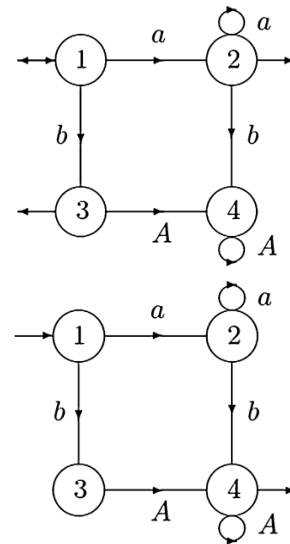
Exemple:

- $A = \{a, b\}$;
- $Q_1 = \{1, 2, 3\}, Q_2 = \{4, 5, 6, 7\}$;
- $D_1 = \{1\}, D_2 = \{4\}$;
- $F_1 = \{2\}, F_2 = \{5\}$;
- $\delta_1 = \{(1, a, 2), (1, b, 3), (2, a, 2), (2, b, 3), (3, a, 3), (3, b, 3)\}$;
- $\delta_2 = \{(4, a, 5), (4, b, 6), (5, a, 6), (5, b, 7), (6, a, 7), (6, b, 5), (7, a, 7), (7, b, 7)\}$.
- $Q_1 \times Q_2 = \{14, 15, 16, 17, 24, 25, 26, 27, 34, 35, 36, 37\}$ (ici, pour simplifier l'écriture, l'état (p, q) est représenté par pq);
- $D = \{14\}$;
- $F = \{25\}$;
- $F' = \{24, 25, 26, 27, 15, 35\}$;
- La table de transitions de ces deux automates est la même :

δ	14	15	16	17	24	25	26	27	34	35	36	37
a	25	26	27	27	25	26	27	27	35	36	37	37
b	36	37	35	37	36	37	35	37	36	37	35	37

1.5.6 Complémentaire d'un automate

1. **Déterminiser** l'automate. (i.e avoir qu'un seul chemin possible lorsqu'on lit un symbole).
2. **Compléter** l'automate. (i.e ajouter un état puit/poubelle dans lequel on met toutes les transitions manquantes).
3. **Inverser** le statut des états. (i.e les états non finals deviennent des états finals et vice versa).



2 Grammaire (ou Langage algébrique de N. Chomsky)

Les automates ne peuvent pas reconnaître tous les langages comme celui qui reconnaît les expressions bien parenthésées ou $\{a^n b^n \mid n \in \mathbb{N}\}$. Il est donc nécessaire d'introduire la notion de grammaire (hors contexte) et d'automate à une pile (AUP).

- Une grammaire est donnée par un quadruplet $G = (V, \Sigma, P, S)$, où:
 - V est l'ensemble des symboles **non terminaux** (en majuscules).
 - Σ est l'ensemble des symboles **terminaux** (en minuscules).
 - $P \subseteq V \times (V \cup \Sigma)^*$ est l'ensemble des **règles de dérivation**.
 - $S \in V$ est le symbole initial, nommée **Seed**.
Remarque: La seed est un symbole non terminal.

- Exemple 1:

PHRASE $\rightarrow$ SUJET VERBE COMPLEMENT
 SUJET $\rightarrow$ LUDOVIC | MICHEL | NICOLAS | THIERRY
 VERBE $\rightarrow$ VOIT | MANGE | PORTE
 COMPLEMENT $\rightarrow$ ARTICLE NOM ADJECTIF
 ARTICLE $\rightarrow$ UN | LE
 NOM $\rightarrow$ LIVRE | PLAT | WAGON
 ADJECTIF $\rightarrow$ BLEU | ROUGE | VERT

- Exemple 2: Pour $L = \{a^n b^n \mid n \in \mathbb{N}\}$. On considère $G = (V, \Sigma, P, S)$ où $V = \{S\}$, $\Sigma = \{a, b\}$, et les règles/productions de G données par

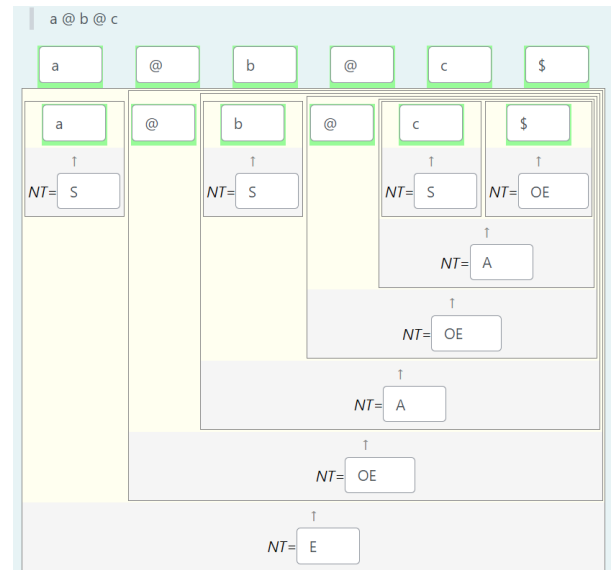
$$S \rightarrow aSb \mid \varepsilon$$

- Exemple 3: Pour le **Langage de Dyck** (= langage qui reconnaît les mots bien parenthésés), on considère l'alphabet $\Sigma = \{a_1, a_1, \dots, a_n, a_n\}$, la grammaire $G = (V, \Sigma, P, S)$ où $V = \{S, T\}$ et les productions de P données par

$$\begin{aligned} S &\rightarrow ST \mid \varepsilon \\ T &\rightarrow a_1Sa_1 \mid \dots \mid a_nSa_n. \end{aligned}$$

- Langage d'une grammaire: $\mathcal{L}(G) = \mathcal{P}(G) \cap \Sigma^*$
(= ensemble des mots qu'on peut former avec la grammaire).
- Remarque: il n'y a aucun non terminal dans le langage d'une grammaire. Ex: $\{(), (()), ()(), ()(), ((())()), \dots\}$

Remarque: il n'y a aucun non terminal dans le langage d'une grammaire. Ex:



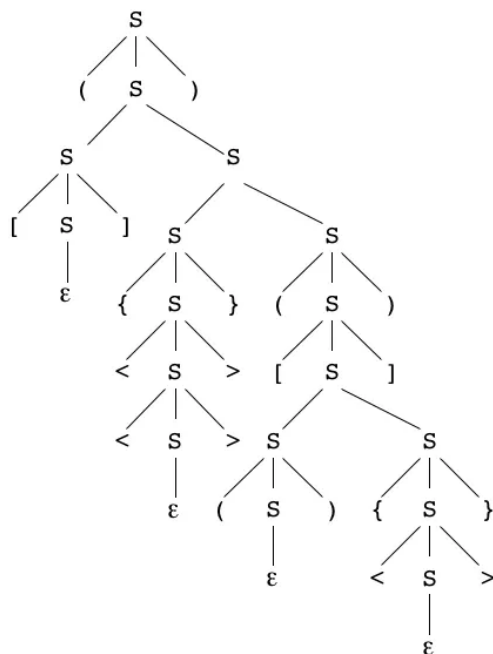
3 Parser, JavaCC, Makefile

...

- L'Arbre d'analyse est un arbre montrant la dérivation d'un mot pour une grammaire dont la racine est la Seed et les feuilles sont les terminaux.

4 Divers

- Livre
- Cours
- Anciens Annales
- Ressources intéressantes
- Utilise le **Lemme d'Arden** pour passer d'un automate a une expression régulière.
 1. Ecrire le système d'équation
 2. Et le remonter avec le lemme d'Arden : Si $L = aL + b$ alors $L = a * b$
- Minimiser un automate avec l'**algorithme de Mac Naughton et Yamada**. (+Moore)
- Construire un AFN à partir d'une expression régulière/rationnelle avec l'**algorithme de Thompson/Glushkov**. **Explication**
- Méthode pour supprimer une récursivité à gauche
- Automate de Thompson



- $E \xrightarrow{1} S.OE$
- $OE \xrightarrow{2} -.E \mid @.A \mid \varepsilon$
- $A \xrightarrow{3} S.OE$
- $S \xrightarrow{4} a \mid b \mid c$

- Livre sympa à lire:

Patrice Séébold



Fondamentaux de la théorie des automates

De l'intuition aux méthodes formelles
en 400 exercices corrigés

